LightningRL: Breaking the Accuracy–Parallelism Trade-off of Block-wise dLLMs via Reinforcement Learning

Yanzhe Hu [1 2] **Yijie Jin** [1] **Pengfei Liu** [1] **Kai Yu** [1] **Zhijie Deng** [1]

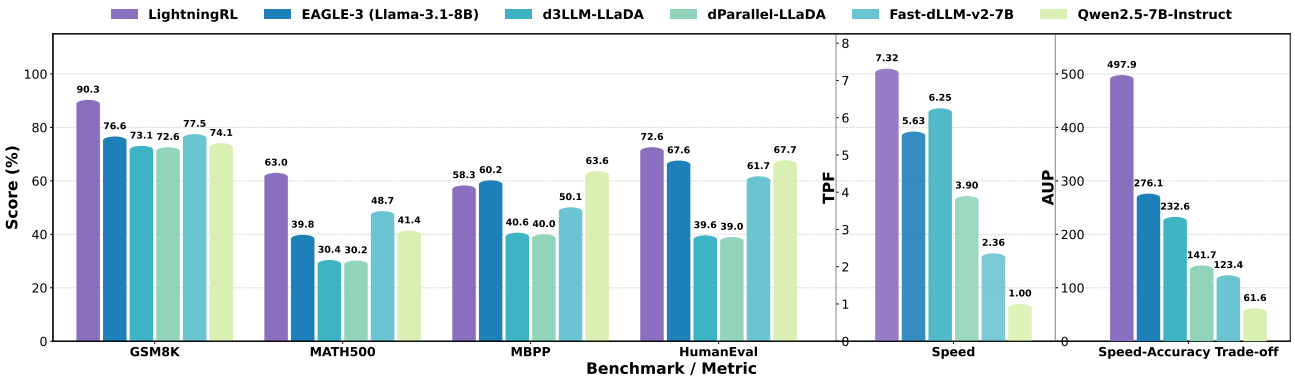


Figure 1. **Evaluation results of LightningRL and the baselines.** LightningRL achieves superior accuracy on math and code benchmarks while accelerating parallel decoding to an average of 7.32 tokens per forward (TPF) and 497.9 accuracy under parallelism (AUP) (Qian et al., 2026), significantly outperforming baselines including Eagle-3 (Li et al., 2025) and Fast-dLLM-v2 (Wu et al., 2025a).

Abstract

Diffusion Large Language Models (dLLMs) enable parallel token generation, and their block-wise variants have attracted significant attention. However, existing dLLMs usually exhibit an accuracy–parallelism trade-off, where raising tokens per forward (TPF) via aggressive parallel decoding often degrades task accuracy. To address this, we suggest developing a post-training approach to directly optimize the speed–quality frontier of pre-trained dLLMs. Conceptually, we do not require the model to decode aggressively along all sampling trajectories, but rather to find several highly parallelizable ones that can yield correct results. To this end, we resort to a reinforcement learning paradigm, i.e., LightningRL, to optimize rewards regarding both the final accuracy and inference parallelism. LightningRL follows the Group Relative Policy Optimization (GRPO) framework, with further improvements for dLLMs: 1) stabilized training via per-reward decoupled normalization, 2) token-level negative log-likelihood (NLL) loss on correct trajectories for regularization, and 3) improved training efficiency through dynamic sampling with TPF-aware filtering. Across maths and code tasks, LightningRL consistently advances the Pareto frontier, maintaining competitive accuracy while increasing parallelism to an average TPF of 7.32 (up to 11.10 on MBPP). The code is now available at https://github.com/SJTU-DENG-Lab/LightningRL.

Work done during Yanzhe Hu's internship at Shanghai Jiao Tong University. [1]Shanghai Jiao Tong University [2]Huazhong University of Science and Technology. Correspondence to: Zhijie Deng <zhijied@sjtu.edu.cn>.

Preprint. March 4, 2026.

1. Introduction

Diffusion Large Language Models (dLLMs) are a promising alternative to autoregressive (AR) decoders for high-throughput generation (Li et al., 2022; Sahoo et al., 2024a; Gulrajani & Hashimoto, 2023; Gong et al., 2023; Sahoo et al., 2024b; Nie et al., 2025; Ye et al., 2025). They frame the generation as iterative denoising of a corrupted token sequence, enabling bidirectional context aggregation and parallel refinement over many positions. However, vanilla dLLMs can suffer from the incompatibility with KV cache mechanisms and fixed generation length.

Block-wise dLLMs (Arriola et al., 2025) address these by bridging AR and diffusion LLMs. They generate blocks of text tokens sequentially to ensure long-range coherence (AR-

like), while simultaneously refining all tokens within each block via diffusion to unlock intra-block parallelism. This approach mitigates the inefficiency of independent token sampling in pure diffusion and the lack of parallelism in AR. In practice, representative works construct block-wise dLLMs by adapting pretrained AR models for block-wise denoising (Cheng et al., 2025; Wu et al., 2025a).

However, existing block-wise dLLMs still suffer from a severe accuracy–parallelism trade-off (Qian et al., 2026), where raising tokens per forward (TPF) via aggressive parallel decoding often degrades task accuracy. Despite training-free sampling strategies (Wu et al., 2025b; Xu et al., 2025) and distillation-based approaches (Wang et al., 2025a; Chen et al., 2025) can boost the TPF and hence the tokens per second (TPS) during inference, it typically comes at the cost of substantial degradation in generation quality. As a result, existing dLLMs are still inferior to popular acceleration approaches to AR LLMs like speculative decoding (Leviathan et al., 2023; Li et al., 2025; Kou et al., 2024) when simultaneously considering speed and quality.

To address this, we advocate a post-training for pre-trained dLLMs that directly optimizes the speed–quality frontier. Our core insight is that we do not require the model to aggressively decode all possible paths; instead, we merely need the model to reliably navigate the specific subspace of trajectories that are both highly parallelizable and accurate. We formulate this objective as a reinforcement learning (RL) problem, using supervision from outcome accuracy and overall TPF to shape the model's probability mass. We implement the resulting **LightningRL** upon the widely-used Group Relative Policy Optimization (GRPO) framework (Shao et al., 2024).

LightningRL makes necessary modifications to GRPO, including 1) *Decoupled Reward Normalization*, which addresses the scale discrepancy between accuracy and TPF rewards via independent normalization to ensure stable multi-objective optimization, 2) *Likelihood-Anchored Regularization*, where a token-level negative log-likelihood (NLL) objective computed on correct trajectories is leveraged to mitigate reward hacking and stabilize updates, and 3) *TPF-aware Filtering*, which selects prompts with sampling trajectories of diverse levels of parallelism to maintain distinct learning signals and improve sample efficiency.

Empirically, we perform LightningRL post-training on the representative block-wise dLLMs SDAR (Cheng et al., 2025) and validate on a comprehensive suite of math and code benchmarks. We report accuracy, TPF (as a measure of parallelism), and accuracy under parallelism (AUP) (Qian et al., 2026), which summarizes the speed–quality trade-off under parallel decoding. The results highlight a superior trade-off profile: our LightningRL-8B model with a block size of 32, tuned from the SDAR-8B, achieves 497.9 AUP with an average speed of 7.32 TPF (up to 11.10 TPF on MBPP). This performance substantially surpasses established acceleration baselines such as d3LLM (Qian et al., 2026) and EAGLE-3 (Li et al., 2025), demonstrating the ability of LightningRL to effectively break the accuracy–parallelism bottleneck of dLLMs.

2. Preliminaries

2.1. Diffusion Large Language Models (dLLMs)

Given an input prompt q, a dLLM generates the response as a Markov Decision Process (MDP) (Li et al., 2022). Ideally, this process produces a trajectory of intermediate states $\{x_0, x_1, \ldots, x_T\}$, where each $x_t \in (\mathcal{V} \cup \{[\text{MASK}]\})^L$ represents the partially decoded sequence at step t. Here, $\mathcal{V}$ denotes the vocabulary, L is the sequence length, and x_0 is the initial fully masked state.

Let $p_\theta(\cdot \mid x_t, q)$ denote the generation distribution parameterized by the model, which effectively serves as the transition policy. The generation proceeds by iteratively sampling the next state based on the current context:

$$x_{t+1} \sim p_\theta(x_{t+1} \mid x_t, q). \tag{1}$$

This aligns with the Markov property, where the next state depends only on the current state x_t and the condition q.

Confidence-driven Decoding (Wu et al., 2025b; Wang et al., 2025a) specifies the transition dynamics by allowing multiple tokens in x_t to be accepted (decoded) in a single iteration if their prediction confidence exceeds a threshold, while rejected tokens are reset to [MASK] in x_{t+1}.

Block Diffusion Vanilla dLLMs suffer from the incompatibility with KV cache mechanisms and fixed generation length (Nie et al., 2025; Ye et al., 2025). Block-wise dLLMs (Arriola et al., 2025; Cheng et al., 2025) address these by partitioning the sequence x into B contiguous blocks $\{x^1, \ldots, x^B\}$, where blocks are generated sequentially while tokens within each block are decoded in parallel. Block-wise dLLMs can be efficiently adapted from pre-trained AR LLMs, with SDAR (Cheng et al., 2025) and Fast-dLLM-v2 (Wu et al., 2025a) as popular examples.

2.2. GRPO for dLLMs

Group Relative Policy Optimization (GRPO) (Shao et al., 2024) has demonstrated remarkable effectiveness for the reinforcement learning (RL) of AR LLMs. Prior works have successfully adapted this paradigm to the non-autoregressive decoding of dLLMs (Wang et al., 2025b; Zhu et al., 2026). To align the RL objective with the dLLM generation process, we explicitly map the policy π_θ to the conditional denoising distribution $p_\theta(\cdot \mid x_t, q)$, and the trajectory τ to the sequence of intermediate noisy states $\{x_t\}$.

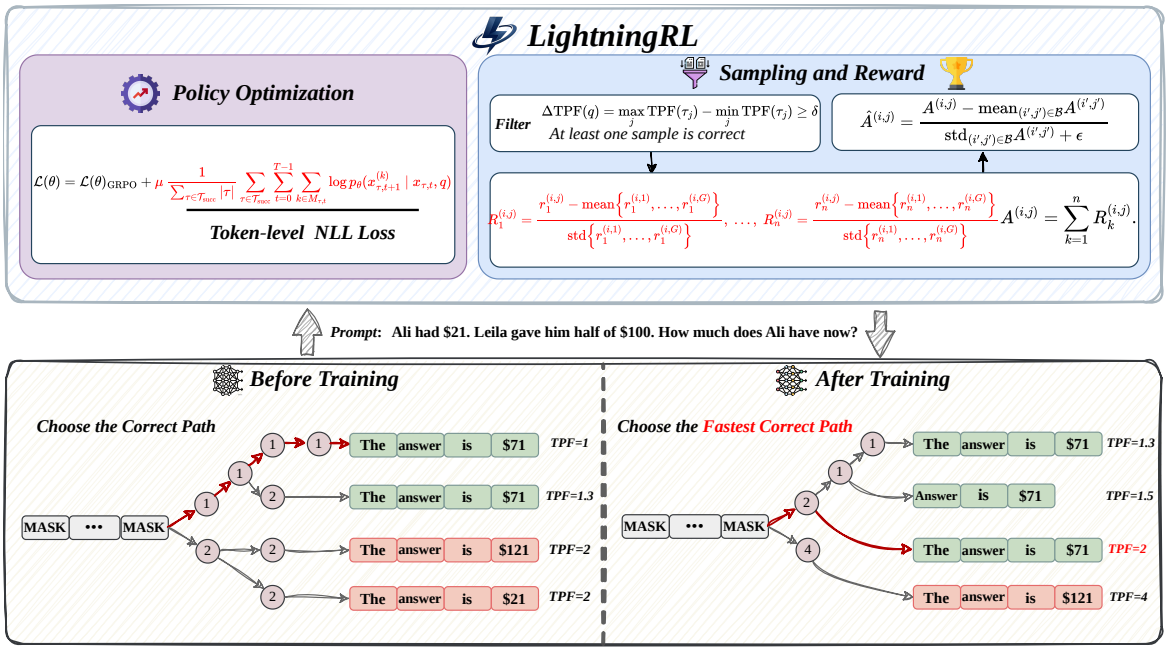


Figure 2. **Overview of LightningRL.** LightningRL samples a group of decoding trajectories per prompt, applies per-reward decoupled normalization to preserve within-group ranking under heterogeneous scales. The policy is optimized with a GRPO-style objective plus a token-level NLL anchor. The bottom panel shows the resulting shift toward the fastest correct trajectory, improving TPF without degrading accuracy.

Specifically, for each prompt q, we sample a group of G rollouted trajectories $\{\tau_j\}_{j=1}^{G}$ using the behavior policy $\pi_{\theta_{\text{old}}}$. GRPO computes the terminal reward $R(\tau_j)$ and derives the advantage $\hat{A}_j$:

$$\hat{A}_j = \frac{R(\tau_j) - \frac{1}{G}\sum_{i=1}^{G} R(\tau_i)}{\text{std}(\{R(\tau_i)\}_{i=1}^{G}) + \epsilon}, \tag{2}$$

where ϵ is a small constant for numerical stability.

GRPO maximizes the advantage-weighted probability of valid actions. Substituting our dLLM-specific policy definition, the loss function is formulated as:

$$\begin{aligned}\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\Bigg[\frac{1}{G}\sum_{j=1}^{G}\frac{1}{|\tau_j|}\sum_{t=0}^{T-1}\sum_{k\in M_{j,t}}\Bigg(& \\ \min\Big(\rho_{j,t,k}\hat{A}_j,\ \text{clip}(\rho_{j,t,k}, 1-\epsilon, 1+\epsilon)\hat{A}_j\Big) & \\ - \beta D_{\text{KL}}\big(p_\theta(\cdot|x_{j,t})\|p_{\text{ref}}(\cdot|x_{j,t})\big)\Bigg)\Bigg]. & \end{aligned} \tag{3}$$

Here, $M_{j,t} = \{i \mid x_{j,t}^{(i)} = \texttt{[MASK]}\}$ denotes the set of indices in the intermediate state $x_{j,t}$ that require denoising, consistent with the generation process defined in Eq. 1. Accordingly, $|\tau_j| = \sum_{t=0}^{T-1}|M_{j,t}|$ represents the total number of parallel token predictions performed along the trajectory. β is the coefficient for the KL divergence penalty against the reference model p_{ref}. The importance ratio $\rho_{j,t,k}$ is:

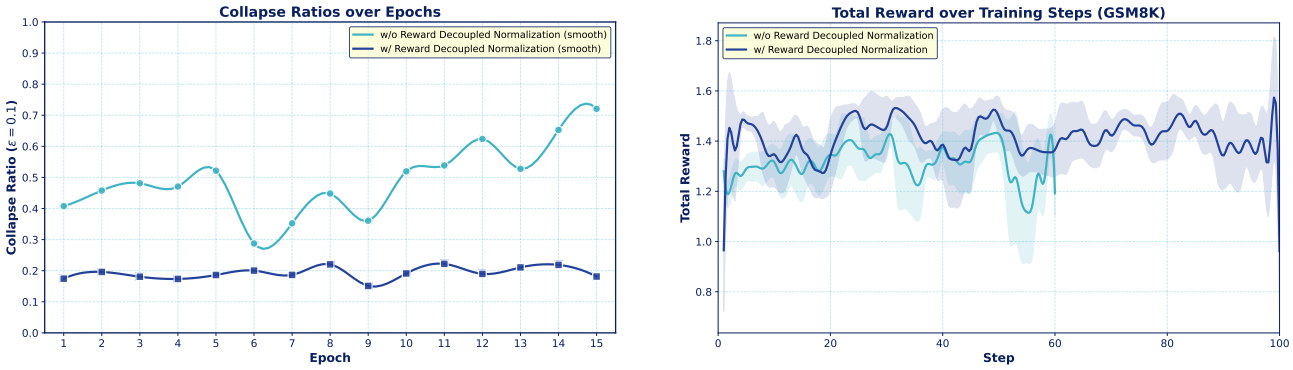
$$\rho_{j,t,k} = \frac{p_\theta(x_{j,t+1}^{(k)} \mid x_{j,t}, q)}{p_{\theta_{\text{old}}}(x_{j,t+1}^{(k)} \mid x_{j,t}, q)}. \tag{4}$$

The components marked in red highlight the structural differences from standard AR-LLMs — gradients are propagated through parallel masked positions over multiple denoising steps conditioned on the intermediate noisy states, rather than sequential token positions conditioned on history.

3. LightningRL: Breaking the Accuracy–Parallelism Trade-off

To push the frontier of dLLMs through RL, we initially adopted existing RL frameworks established for dLLMs (Zhao et al., 2025; Wang et al., 2025b; Zhu et al., 2026). However, we encountered significant challenges due to our multi-objective setting (i.e., simultaneously optimizing for accuracy and speed). We observed reward collapse, where one objective dominates the optimization. Thus, the policy drifts, and the model's generation capability degrades.

To address these, we present LightningRL, a robust training algorithm designed to co-optimize accuracy and inference parallelism. The overall architecture of LightningRL is



(a) Collapse ratio across epochs on GSM8K (lower is better).

(b) Total reward during training on GSM8K.

Figure 3. **Per-reward decoupled normalization improves training stability.** It reduces signal collapse (a) and yields more stable reward optimization (b) under the same training setup.

illustrated in Fig. 2, which incorporates three specific modifications designed for the dLLM landscape:

- Per-reward Decoupled Normalization (Sec. 3.1), which mitigates reward collapse by independently normalizing distinct reward signals.
- Token-level NLL Regularization (Sec. 3.2), which is applied to correct trajectories to prevent policy drift and maintain linguistic coherence.
- Dynamic Sampling with TPF-aware Filtering (Sec. 3.3), which helps efficiently explore the trade-off between speed and quality.

3.1. Decoupled Normalization for Group Rewards

While GRPO is effective for single-objective optimization, a naive extension to multi-reward settings, which involves summing rewards and subsequently applying standard group-wise normalization, tends to be suboptimal. When a coarse, high-magnitude discrete reward (e.g., $\mathrm{Acc} \in \{-1, 1\}$) is combined with a fine-grained continuous reward (e.g., the overall TPF of a sampling trajectory), the aggregate value is frequently dominated by the discrete term, resulting in numerous within-group ties or near-ties. This phenomenon renders advantages indistinguishable across different behaviors and consequently diminishes the strength of the effective policy-gradient signal.

To mitigate this issue, we propose per-reward decoupled normalization. Instead of normalizing the aggregated reward, we normalize each reward component independently within the group and then aggregate the standardized signals. Consider the i-th prompt in the current minibatch and its generated group of rollouts of size G. For the j-th rollout, let $r_k^{(i,j)}$ denote the raw value of the k-th reward objective among a total of n objectives. Instead of summing r_k, we first compute the independent advantage for each objective:

$$R_k^{(i,j)} = \frac{r_k^{(i,j)} - \mathrm{mean}\left\{r_k^{(i,1)}, \ldots, r_k^{(i,G)}\right\}}{\mathrm{std}\left\{r_k^{(i,1)}, \ldots, r_k^{(i,G)}\right\} + \epsilon}, \tag{5}$$

where ϵ is a small constant for numerical stability. This ensures that every reward component contributes an equally standardized ranking signal, regardless of its raw magnitude or distribution. However, during the actual training process, we do not normalize the accuracy reward to prevent reward shifts. The composite advantage is then derived by summing these normalized components:

$$A^{(i,j)} = \sum_{k=1}^{n} R_k^{(i,j)}. \tag{6}$$

Finally, we apply global batch-wise normalization to control the update scale regardless of the number of objectives, yielding the final advantage $\hat{A}^{(i,j)}$ used for policy updates:

$$\hat{A}^{(i,j)} = \frac{A^{(i,j)} - \mathrm{mean}_{(i',j')\in\mathcal{B}} A^{(i',j')}}{\mathrm{std}_{(i',j')\in\mathcal{B}} A^{(i',j')} + \epsilon}, \tag{7}$$

where $\mathcal{B}$ denotes the set of all rollout indices (i', j') in the current minibatch. Crucially, while the preceding decoupling step preserves group-relative distinctions contributed by each objective, this final normalization prevents the advantage magnitude from drifting with the reward dimensionality n.

To validate our motivation, we present two diagnostics. First, Fig. 3a shows that decoupled normalization substantially reduces the Collapse Ratio, defined as the proportion of within-group pairs with advantage differences below ϵ, thereby preserving finer preference resolution. Second, as shown in Fig. 3b, removing this decoupling step leads to noisier optimization trajectories, slower convergence, and

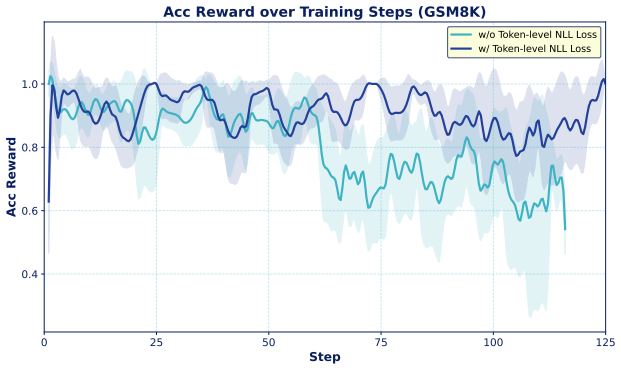


Figure 4. **Token-level NLL loss anchors the accuracy objective on GSM8K.** Compared with training without the token-level NLL term, it maintains a higher accuracy reward and mitigates late-stage drift in the accuracy signal under the same setup.

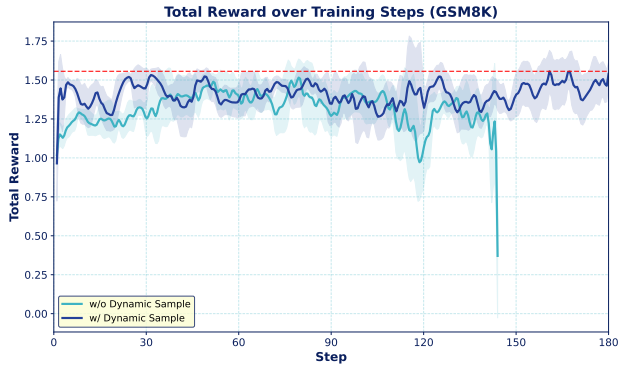


Figure 5. **Dynamic sampling improves robustness of total-reward optimization on GSM8K.** Relative to training without dynamic sampling, it reduces reward collapse under the same setup.

lower peak performance. These results confirm that decoupling improves the conditioning of aggregated advantages by maintaining meaningful distinctions while stabilizing the update scale.

3.2. Token-Level NLL for Accuracy Anchoring

In multi-objective RL with verifier-based supervision, rewards on math and code tasks are typically sparse and sequence-level. In this setting, the policy gradient estimator necessarily takes a score-function form, where a scalar advantage multiplies log-probability gradients of the actions taken along the trajectory.

Formally, GRPO assigns each sampled trajectory τ a single group-relative advantage $\hat{A}(\tau)$. Abstracting away the piecewise clipping details, the induced policy update admits the canonical score-function structure:

$$\nabla_\theta \mathcal{L}_{\text{Policy}} \propto -\mathbb{E}\Bigg[\frac{\hat{A}(\tau)}{|\tau|}\sum_{t=0}^{T-1}\sum_{k\in M_{\tau,t}} \nabla_\theta \log p_\theta(x^{(k)}_{\tau,t+1} \mid x_{\tau,t}, q)\Bigg] \tag{8}$$

Crucially, $\hat{A}(\tau)$ is shared by all decoding steps in a trajectory: the factor $1/|\tau|$ only rescales the update and does not refine credit assignment. In multi-objective training, the *accuracy-driven* gradient can be substantially attenuated by per-token dilution and further overshadowed when non-accuracy rewards dominate the effective advantage. This weakens the corrective pressure toward correctness and can induce drift into fast-but-incorrect modes. This motivates a positive-example LM loss that turns verifier-correct trajectories into dense, token-factorized anchoring toward correct behaviors.

To maximize the utility of rare verified successes and explicitly anchor the policy toward correctness, we introduce a token-level NLL loss that converts sequence-level successes into dense token-factorized supervision. Let $\mathcal{T}_{\text{succ}}$ denote the set of verifier-correct trajectories among on-policy samples in the current update. We define:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{\sum_{\tau\in\mathcal{T}_{\text{succ}}}\sum_{t=0}^{T-1}|M_{\tau,t}|}\sum_{\tau\in\mathcal{T}_{\text{succ}}} \sum_{t=0}^{T-1}\sum_{k\in M_{\tau,t}} \log p_\theta(x^{(k)}_{\tau,t+1} \mid x_{\tau,t}, q), \tag{9}$$

and set $\mathcal{L}_{\text{NLL}}(\theta) = 0$ when $\mathcal{T}_{\text{succ}} = \emptyset$. Here, consistent with Sec. 2.1, $M_{\tau,t}$ denotes the masked indices at step t for trajectory τ. Overall, we optimize the combined objective:

$$\mathcal{L}_{\text{LightningRL}}(\theta) = \underbrace{\mathcal{L}_{\text{Policy}}(\theta) + \beta\,\mathcal{L}_{\text{KL}}(\theta)}_{\mathcal{L}_{\text{GRPO}}(\theta)} + \mu\,\mathcal{L}_{\text{NLL}}(\theta), \tag{10}$$

where β and μ are scalar coefficients balancing the loss components. This formulation yields a clean division of labor: $\nabla_\theta\mathcal{L}_{\text{GRPO}}$ drives preference learning via relative ranking across sampled trajectories, while $\mu\nabla_\theta\mathcal{L}_{\text{NLL}}$ acts as a self-imitation anchor that reallocates probability mass onto verifier-correct trajectories with stable, token-factorized gradients. Fig. 4 supports this anchoring effect: while training without token-level NLL loss exhibits a pronounced downward drift, the anchored variant maintains consistently higher accuracy rewards and substantially improved stability over training steps.

3.3. Dynamic Sampling for Efficient Policy Optimization

Standard GRPO can suffer from gradient starvation under reward quantization. GRPO estimates advantages based on within-group relative rewards; consequently, when the G sampled trajectories $\{\tau_j\}_{j=1}^{G}$ for a prompt q fall into the

Table 1. **Evaluation results of LightningRL and methods on math and code benchmarks.** The notation *LightningRL-8B-b32* denotes the 8B LightningRL model with a block size of 32. *GRPO(traj)* refers to the method introduced in Sec. 2.2, which can be viewed as TraceRL without a value model and with group-wise advantage estimation. Compared with the other two algorithms and the SDAR base model, LightningRL delivers substantial improvements across both math and code benchmarks.

Model	GSM8K			MATH500			MBPP			HumanEval		
	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP
SDAR-8B-b32 (Cheng et al., 2025)	88.9	2.85	252.5	**63.6**	4.81	299.5	58.0	2.44	81.1	73.5	2.39	123.8
TraceRL-8B-b32 (Wang et al., 2025b)	76.9	5.04	378.6	60.6	4.82	284.6	57.8	2.50	144.2	75.0	2.29	171.6
GRPO(traj)-8B-b32	86.6	4.87	414.1	61.6	5.04	301.2	56.8	2.70	152.7	**76.0**	2.33	176.7
LightningRL-8B-b32	**90.3**	**5.58**	**492.4**	63.0	**6.28**	**407.5**	**58.3**	**11.10**	**641.6**	72.6	**6.30**	**450.1**

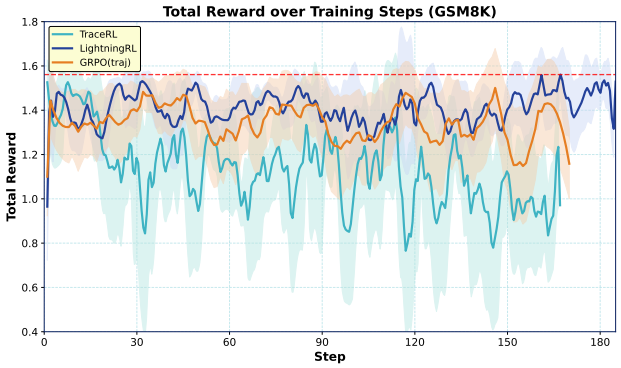


Figure 6. **Total Reward curves over training steps.** LightningRL outperforms baseline methods by suggesting a more reliable optimization landscape.

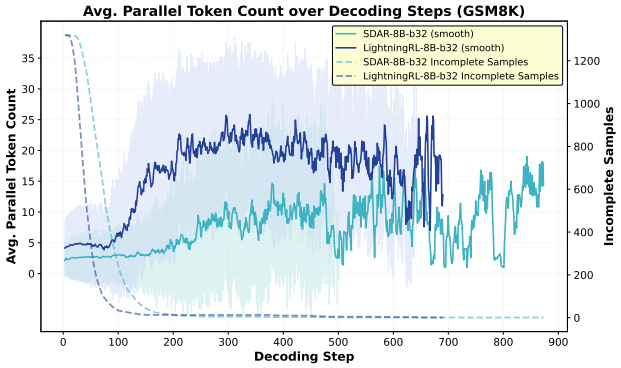


Figure 7. **Decoding behavior.** LightningRL maintains higher throughput and reduces long-tail decoding steps relative to the baseline.

same reward bin, the relative advantages vanish. This issue is particularly pronounced in our multi-objective setting: once the accuracy reward saturates within a group, differentiation relies solely on the speed reward, which is derived from discrete TPF (Token Passing Fraction) values. As a result, many groups yield identical total rewards $R(\tau_j)$, leading to vanishing gradients and wasted rollout budgets. Furthermore, if all samples within a group are incorrect, the total reward becomes entirely dominated by the speed reward, which often results in reward hacking.

To mitigate this issue, we propose dynamic sampling with TPF-aware filtering. We define the within-group TPF spread as:

$$\Delta\mathrm{TPF}(q) = \max_j \mathrm{TPF}(\tau_j) - \min_j \mathrm{TPF}(\tau_j) \geq \delta, \quad (11)$$

where δ is a predefined filtering threshold. For each candidate prompt, we first sample a group of trajectories and accept the prompt only if it satisfies the aforementioned criterion and at least one sample is correct; otherwise, we discard it and resample a new prompt. We repeat this accept–reject process until the batch is filled. By filtering out near-tie groups, this procedure keeps a more consistent fraction of non-zero advantages in each update, leading to denser and more stable policy gradient signals.

Empirically, Fig. 5 shows that without dynamic sampling the training curve converges more slowly, reaches a lower plateau, and eventually undergoes a pronounced collapse, whereas with dynamic sampling training remains stable and achieves faster, higher-reward convergence under the same configuration.

4. Experiments

4.1. Setup

Model and Dataset We implement LightningRL upon the SDAR model family (Cheng et al., 2025) due to its leading performance. We refer to the 8B SDAR model with a block size of 32 as SDAR-8B-b32, with all other variants named likewise. For training, we utilize the training split of the MATH (Hendrycks et al., 2021) dataset and GSM8K (Cobbe et al., 2021) for mathematical reasoning and the PrimeIntellect (Team et al., 2025) dataset for code generation tasks.

Reinforcement Learning Settings During data collection, we use a batch size of 128 tasks and a group size of 32 responses per task. We employ a low confidence dynamic sampling strategy with a threshold $\phi = 0.9$ and a sampling temperature of 1.0. We provide additional training details in the Appendix B.1.

Benchmark and Baselines We evaluate on four representative benchmarks: GSM8K (Cobbe et al., 2021),

Table 2. **Evaluation results of LightningRL and the baselines on math and code benchmarks.** LightningRL consistently advances the speed–quality frontier over its SDAR baseline and prior diffusion and autoregressive baselines, achieving substantially higher AUP at comparable accuracy.

Model	GSM8K			MATH500			MBPP			HumanEval		
	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP
dLLMs												
Dream (Ye et al., 2025)	83.9	1.00	83.9	39.6	1.00	39.6	57.2	1.00	57.2	55.2	1.00	55.2
Fast-dLLM-Dream (Wu et al., 2025b)	79.0	1.44	116.5	38.3	1.78	55.2	53.2	1.20	63.6	54.3	1.33	63.5
dParallel-Dream (Chen et al., 2025)	82.1	3.02	245.7	38.7	2.94	77.9	55.4	2.24	108.0	54.3	2.57	98.8
d3LLM-Dream (Qian et al., 2026)	81.4	4.94	391.3	38.2	3.92	97.5	55.6	2.96	141.4	57.1	3.20	129.5
LLaDA (Nie et al., 2025)	72.6	1.00	72.6	32.2	1.00	32.2	47.1	1.00	47.1	38.3	1.00	38.3
Fast-dLLM-LLaDA (Wu et al., 2025b)	74.7	2.77	205.8	30.8	1.97	47.2	38.6	2.13	56.6	37.8	2.56	54.0
D2F-LLaDA (Wang et al., 2025a)	73.2	2.88	209.7	28.7	2.38	45.5	38.0	1.94	50.0	36.6	2.69	62.0
dParallel-LLaDA (Chen et al., 2025)	72.6	5.14	358.1	30.2	3.17	64.5	40.0	2.35	60.5	39.0	4.93	83.7
d3LLM-LLaDA (Chen et al., 2025)	73.1	**9.11**	**637.7**	30.4	5.74	107.6	40.6	4.21	88.4	39.6	5.95	96.6
AR Models												
Qwen-2.5-7B-it (Qwen et al., 2025)	74.1	1.00	74.1	41.4	1.00	41.1	**63.6**	1.00	63.6	67.7	1.00	67.7
EAGLE-3 (LLaMA-3.1) (Li et al., 2025)	76.6	5.12	319.0	39.8	5.72	142.1	60.2	5.69	298.6	67.6	5.98	344.8
Block-wise dLLMs												
Fast-dLLM-v2 (Wu et al., 2025a)	77.5	2.21	156.0	48.7	2.61	126.7	50.1	2.04	81.9	61.7	2.58	128.9
SDAR-8B-b32 (Cheng et al., 2025)	88.9	2.85	252.5	**63.6**	4.81	299.5	58.0	2.44	81.1	**73.5**	2.39	123.8
LightningRL-8B-b32	**90.3**	5.58	492.4	63.0	**6.28**	**407.5**	58.3	**11.10**	**641.6**	72.6	**6.30**	**450.1**

MATH500 (Hendrycks et al., 2021), HumanEval (Chen et al., 2021), and MBPP (Austin et al., 2021). To ensure a fair comparison with prior work, we employ a 4-shot setting for MATH and a 3-shot setting for Llada-based models (Nie et al., 2025) on MBPP. All other evaluations are conducted in a zero-shot setting. We benchmark our approach against state-of-the-art dLLMs and AR models.

Evaluation Metrics We assess performance based on three key evaluation metrics: TPF for parallelism, accuracy, and the AUP (Qian et al., 2026) score. Furthermore, we detail the TPS results for our model in Appendix C.2.

4.2. Main Results

As demonstrated in Tab. 1, LightningRL consistently outperforms GRPO(traj) and TraceRL across four benchmarks, especially in the coding domain. On the MBPP dataset, LightningRL achieves an AUP of 641.6 and a TPF of 11.10. These results substantially surpass TraceRL, which scores 144.2 in AUP and 2.50 in TPF, as well as GRPO(traj) with a TPF of 2.70. Beyond code generation, LightningRL maintains a strong advantage in mathematical reasoning tasks. The framework records AUP scores of 492.4 on GSM8K and 407.5 on MATH500, confirming its broad effectiveness in accelerating convergence and improving overall training efficiency across diverse domains.

4.3. Training Dynamic

As illustrated in Fig. 6, compared to the baseline approaches, LightningRL effectively prevents deviations in the optimization direction across both mathematical and coding tasks. We observe that a deviating optimization process produces widespread negative advantages. While this reduces the probability of undesirable patterns, it concurrently suppresses favorable ones, leading to overall training failure. A comparison between the training curves of TraceRL and GRPO(traj) reveals that training stability improves significantly upon the removal of the value model. A further discussion of this phenomenon is provided in Appendix A, and the training dynamics are detailed in Appendix C.3.

4.4. Analysis of Decoding Behavior

Figure 7 illustrates the step-wise decoding dynamics of the model trained using LightningRL. In comparison to SDAR, LightningRL executes the decoding process across multiple samples with significantly higher parallelism. Specifically, the majority of samples processed by LightningRL terminate within approximately 100 steps, whereas the decoding process of SDAR exhibits a heavy-tail distribution extending beyond 850 steps. This improvement in performance stems from a higher throughput achieved during the active decoding phase. Consequently, LightningRL effectively mitigates the occurrence of long-tail decoding steps.

4.5. Benchmark Result

An illustration of the comparison between LightningRL and leading baselines is provided in Fig. , with a complete comparison with more baselines detailed in Tab. 2. As shown, LightningRL-8B-b32 achieves an average AUP of 497.9 and an average TPF of 7.32 on these baselines, with an average accuracy of 71.1%. In comparison, the SDAR-8B-b32 baseline attains only an average AUP of 189.2 and a TPF of 3.12 while remaining almost identical average accuracy (71.0%). Moreover, LightningRL consistently dominates representa-

Table 3. **Ablation study of LightningRL components on GSM8K.** We present the accuracy, parallelism, and AUP score of the three components at the highest AUP during 20 training epochs. The results show that each component is critical to maintaining the optimal Pareto frontier.

Model	Acc (%)	TPF	AUP
w/o NLL loss	80.7	5.03	385.7
w/o Decoupled normalization	85.3	4.96	416.5
w/o TPF-aware filtering	87.2	5.27	454.5
LightningRL	**90.3**	**5.58**	**492.4**

Table 4. **Ablation study of loss reduction strategies on GSM8K.**We present the accuracy, parallelism, and AUP score on different loss reduction strategies. The notation *Seq-Tok-Tok* denotes the reduction strategy of policy loss, KL loss, and NLL loss.

Reduction Strategy	Acc (%)	TPF	AUP
Seq-Seq-Seq	87.5	5.42	467.1
Seq-Tok-Seq	88.7	4.86	424.7
Seq-Tok-Tok	**90.3**	**5.58**	**492.4**
Tok-Tok-Tok	80.0	3.91	306.5

tive AR and diffusion baselines, surpassing EAGLE-3 (Li et al., 2025) (276.1 average AUP, 5.63 average TPF) and the d3LLM (Qian et al., 2026) family across the same evaluation settings.

4.6. Ablation

Ablation Study on three components. To assess the contribution of each component within LightningRL, we conduct ablation studies on the LightningRL-8B-b32 model using the GSM8K dataset. As shown in Tab. 3, we compare our full method against three variants. Empirical results confirm that all components are integral to LightningRL's peak performance of 90.3% accuracy, 5.58 TPF, and 492.4 AUP. Ablations lead to clear performance drops: removing the NLL loss causes the most substantial accuracy decline to 80.7%, omitting decoupled normalization reduces accuracy to 85.3% and TPF to 4.96, and excluding TPF-aware filtering lowers accuracy to 87.2%.

Ablation Study on different loss reduction strategies. LightningRL optimizes a weighted sum of policy, KL regularization, and NLL losses. We investigate loss reduction strategies by comparing token-level and sequence-level averaging. Token-level reduction weights all tokens globally, while sequence-level reduction averages within each sequence to weight rollouts equally. These choices induce distinct implicit sample weightings under varying sequence lengths, substantially altering training dynamics. As Tab. 4 demonstrates, the Seq-Tok-Tok configuration achieves an optimal accuracy and AUP, whereas pure token-level reduction degrades accuracy to 80.0%. Prior work (Ou et al., 2025) attributes this degradation to the mismatch between autoregressive objectives and non-autoregressive diffusion processes. Since diffusion models lack tractable token-level conditional likelihoods, current methods rely on heuristic proxies that introduce bias and inconsistency during token-level optimization.

5. Related Work

5.1. dLLM Acceleration

dLLMs offer a non-autoregressive generation paradigm that iteratively denoises a fully or partially masked sequence, enabling parallel token updates but leaving practical inference efficiency still comparatively underexplored. Recent acceleration work largely falls into: (i) reducing per-step compute via diffusion-compatible caching and confidence-aware decoding rules (Wu et al., 2025b;a); (ii) increasing effective parallelism and/or cutting denoising steps through improved unmasking, sampling, and learned parallel decoding strategies (Xu et al., 2025; Bao et al., 2025; Chen et al., 2025); and (iii) hybrid diffusion–autoregressive pipelines that better leverage block-wise decoding and KV-cache-style components (Wang et al., 2025a). Our work builds upon these acceleration techniques but focuses on optimizing the decoding trajectory via reinforcement learning to mitigate the accuracy loss typically associated with high parallelism.

5.2. Reinforcement Learning in dLLMs

Recent advances in dLLMs leverage RL for step-wise denoising. Beyond early masked SFT and policy optimization (Zhao et al., 2025; Gong et al., 2025), research has shifted toward trajectory-level optimization, including trace-aware RL (Wang et al., 2025b), step-aligned scheduling (He et al., 2025), and outcome-driven reasoning (Huang et al., 2025). Recent refinements further incorporate accuracy-aware signals (Ma et al., 2024) or joint control-variable optimization (Zhou et al., 2025). Despite these gains, existing frameworks prioritize generation quality and relegate parallelism to a mere inference-time adjustment. LightningRL addresses this gap by treating parallelism as a first-class training objective, employing multi-objective RL to jointly optimize speed and accuracy. Decoupled and anchored regularization adopted by LightningRL stabilizes multi-objective training against sparse-reward instability.

6. Conclusion

In this paper, we presented LightningRL, an RL framework designed to optimize the speed–quality trade-off in diffusion language models. By mitigating the error amplification inherent in aggressive parallel decoding, LightningRL reconciles the conflict between generation speed and accuracy. Our results on maths and code generation benchmarks demonstrate that LightningRL consistently achieves higher AUP under high parallelism constraints, paving the way for practical, high-throughput dLLM deployment. Future work will explore scaling laws with larger contexts and generalize this approach to a wider range of dLLM generation tasks.

Impact Statement

This paper presents foundational research in deep learning, aiming to advance the field of machine learning. While optimizing the accuracy–parallelism frontier and inference efficiency of Diffusion Large Language Models via reinforcement learning may yield potential societal impacts, we do not engage in a detailed discussion here, as the ethical implications and anticipated societal consequences of developments in efficient generative AI are already well-established.

References

Arriola, M., Gokaslan, A., Chiu, J. T., Yang, Z., Qi, Z., Han, J., Sahoo, S. S., and Kuleshov, V. Block diffusion: Interpolating between autoregressive and diffusion language models, 2025. URL https://arxiv.org/abs/2503.09573.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Bao, W., Chen, Z., Xu, D., and Shang, Y. Learning to parallel: Accelerating diffusion large language models via learnable parallel decoding, 2025. URL https://arxiv.org/abs/2509.25188.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. 2021.

Chen, Z., Fang, G., Ma, X., Yu, R., and Wang, X. dparallel: Learnable parallel decoding for dllms. *arXiv preprint arXiv:2509.26488*, 2025.

Cheng, S., Bian, Y., Liu, D., Zhang, L., Yao, Q., Tian, Z., Wang, W., Guo, Q., Chen, K., Qi, B., and Zhou, B. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation, 2025. URL https://arxiv.org/abs/2510.06303.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gong, S., Li, M., Feng, J., Wu, Z., and Kong, L. Diffuseq: Sequence to sequence text generation with diffusion models, 2023. URL https://arxiv.org/abs/2210.08933.

Gong, S., Zhang, R., Zheng, H., Gu, J., Jaitly, N., Kong, L., and Zhang, Y. Diffucoder: Understanding and improving masked diffusion models for code generation, 2025. URL https://arxiv.org/abs/2506.20639.

Gulrajani, I. and Hashimoto, T. B. Likelihood-based diffusion language models, 2023. URL https://arxiv.org/abs/2305.18619.

He, H., Renz, K., Cao, Y., and Geiger, A. Mdpo: Overcoming the training-inference divide of masked diffusion language models, 2025. URL https://arxiv.org/abs/2508.13148.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Huang, Z., Chen, Z., Wang, Z., Li, T., and Qi, G.-J. Reinforcing the diffusion chain of lateral thought with diffusion language models, 2025. URL https://arxiv.org/abs/2505.10446.

Kou, S., Hu, L., He, Z., Deng, Z., and Zhang, H. Cllms: Consistency large language models. In *Forty-first International Conference on Machine Learning*, 2024.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

Li, X. L., Thickstun, J., Gulrajani, I., Liang, P., and Hashimoto, T. B. Diffusion-lm improves controllable text generation, 2022. URL https://arxiv.org/abs/2205.14217.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025. URL https://arxiv.org/abs/2503.01840.

Ma, Y., Melnychuk, V., Schweisthal, J., and Feuerriegel, S. Diffpo: A causal diffusion model for learning distributions of potential outcomes, 2024. URL https://arxiv.org/abs/2410.08924.

Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models, 2025. URL https://arxiv.org/abs/2502.09992.

Ou, J., Han, J., Xu, M., Xu, S., Xie, J., Ermon, S., Wu, Y., and Li, C. Principled rl for diffusion llms emerges from a sequence-level perspective, 2025. URL https://arxiv.org/abs/2512.03759.

Qian, Y.-Y., Su, J., Hu, L., Zhang, P., Deng, Z., Zhao, P., and Zhang, H. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation, 2026. URL https://arxiv.org/abs/2601.07568.

Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models, 2024a. URL https://arxiv.org/abs/2406.07524.

Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models, 2024b. URL https://arxiv.org/abs/2406.07524.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Team, P. I., Senghaas, M., Obeid, F., Jaghouar, S., Brown, W., Ong, J. M., Auras, D., Sirovatka, M., Straube, J., Baker, A., Müller, S., Mattern, J., Basra, M., Ismail, A., Scherm, D., Miller, C., Patel, A., Kirsten, S., Sieg, M., Reetz, C., Erdem, K., Weisser, V., and Hagemann, J. Intellect-3: Technical report, 2025. URL https://arxiv.org/abs/2512.16144.

Wang, X., Xu, C., Jin, Y., Jin, J., Zhang, H., and Deng, Z. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing, 2025a. URL https://arxiv.org/abs/2508.09192.

Wang, Y., Yang, L., Li, B., Tian, Y., Shen, K., and Wang, M. Revolutionizing reinforcement learning framework for diffusion large language models, 2025b. URL https://arxiv.org/abs/2509.06949.

Wu, C., Zhang, H., Xue, S., Diao, S., Fu, Y., Liu, Z., Molchanov, P., Luo, P., Han, S., and Xie, E. Fast-dllm v2: Efficient block-diffusion llm, 2025a. URL https://arxiv.org/abs/2509.26328.

Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025b. URL https://arxiv.org/abs/2505.22618.

Xu, C., Jin, Y., Li, J., Tu, Y., Long, G., Tu, D., Song, M., Si, H., Hou, T., Yan, J., and Deng, Z. Lopa: Scaling dllm inference via lookahead parallel decoding, 2025. URL https://arxiv.org/abs/2512.16229.

Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models, 2025. URL https://arxiv.org/abs/2508.15487.

Zhao, S., Gupta, D., Zheng, Q., and Grover, A. d1: Scaling reasoning in diffusion large language models via reinforcement learning, 2025. URL https://arxiv.org/abs/2504.12216.

Zhou, R., Ni, Z., Chen, T., Liu, Z., Yue, Y., Wang, Y., Wang, Y., Liu, J., and Huang, G. Co-grpo: Co-optimized group relative policy optimization for masked diffusion model, 2025. URL https://arxiv.org/abs/2512.22288.

Zhu, Y., Wan, J., Liu, X., He, S., Wang, Q., Guo, X., Liang, T., Huang, Z., He, Z., and Qiu, X. Dirl: An efficient post-training framework for diffusion language models, 2026. URL https://arxiv.org/abs/2512.22234.

A. Discussion on Value Model Incorporation

Method	Acc (%)	TPF	AUP
w/ Value Model	80.7	5.14	408.1
LightningRL	**90.3**	**5.58**	**492.4**

Table 5. Comparison of Performance with and without Value Model Incorporation on GSM8K.

To further assess whether a learned critic benefits LightningRL, we integrated a value network V_ϕ and used GAE for advantage estimation. As shown in Tab. 5, this variant yields a clear drop in both accuracy and TPF.

We conjecture that the critic becomes unreliable under the highly non-smooth state transitions induced by block-wise decoding, causing frequent *advantage sign flips*. Let $V_\phi(x_t) = V^\pi(x_t) + \epsilon(x_t)$, where $\epsilon(x_t)$ denotes the value approximation error. Under GAE, the estimated advantage can be written as the true advantage plus an accumulated error term that is a weighted combination of $\epsilon(x_{t+l})$ along the trajectory. When these errors are large relative to the underlying advantage signal (notably in our setting where correctness reward is largely terminal and intermediate signals are weak), the critic-induced noise can dominate and flip the sign of $\hat{A}_t$, leading to updates that penalize beneficial actions or reinforce suboptimal ones.

This issue is exacerbated by confidence-driven block decoding/remasking, where a single step updates many tokens and can induce abrupt deviations in the partial sequence state, making it substantially harder for a scalar value function to learn a coherent and stable baseline over the visited state manifold. We leave a systematic empirical characterization of such critic instability (e.g., correlation between critic-based advantages and return-based proxies) for future work.

B. Experimental Details

B.1. Training Details

We present the hyperparameters and experimental configurations utilized in our study. For the SDAR models, we employ a default block size of 32, dynamic decoding with a threshold of $\phi = 0.9$, top-$k = 0$, top-$p = 1$, and a temperature of 1.0. Regarding the rollout configuration, we utilize a block size of 32 with 32 denoising steps per block during the diffusion-based generation process. The maximum response length is set to 8192. In each training iteration, we sample 128 tasks and generate 32 responses per task. We use a temperature of 1.0 for exploration during training, while greedy decoding is applied for evaluation. Furthermore, we employ low-confidence dynamic remasking with a confidence threshold of 0.9 to selectively re-mask low-probability tokens during the denoising steps.

In terms of reward design, we adopt a formulation that incorporates both accuracy and speed signals. For mathematics tasks, we use binary outcomes as verifiable rewards based on answer equivalence checking, assigning a base reward of ± 1.0 for correct or incorrect answers. We apply configurable filtering to retain only tasks where at least one response is correct and where the token-per-forward (TPF) variance exceeds a specific threshold (defaulting to 0.01). For coding tasks, we utilize the proportion of unit tests passed by the generated solutions as the reward. The speed reward is computed based on the TPF. Subsequently, we apply per-reward decoupled normalization.

The training procedure varies depending on the model components used. When a value model is incorporated, it is optimized using the mean squared error (MSE) loss on token-level returns. In such configurations, we select $(\gamma, \lambda) = (1, 1)$ and employ generalized advantage estimation (GAE) with $\lambda = 1.0$ for both pre-training and main training. The policy model employs a PPO-style clipped surrogate objective with $\epsilon = 0.2$. We utilize the KL divergence estimator $k = 3$ with a coefficient of $\beta = 0.01$, computed against a frozen reference model. Additionally, a negative log-likelihood (NLL) loss with a weight of 0.1 is applied to correct samples. The optimization process is performed using AdamW, with a learning rate of 1×10^{-6} for the policy and 5×10^{-6} for the value function (if applicable). We employ a constant learning rate scheduler with 10 warmup steps, a maximum gradient norm of 1.0, and gradient checkpointing. To accelerate the process, we implement distributed training across multiple GPU nodes; our experiments utilize configurations such as a single node equipped with 8 H200 GPUs, leveraging DeepSpeed ZeRO-1 optimization. Finally, for data processing, we employ sequential sampling with cursor-based traversal through the training dataset, which automatically reshuffles after the completion of a full pass.

B.2. LightningRL Algorithm Pipeline

Algorithm 1 LightningRL

Input:
1) Training set $\mathcal{D} = \{q, a\}$ of prompts and reference answers
2) Policy π_θ; frozen reference policy π_{ref}
3) Iterations M; target accepted prompt-groups per iteration B; rollouts per prompt G
4) PPO clips ϵ; update epochs K; optimizer lr η
5) Reward/penalty weights: β_{KL}, w_{NLL}
6) Filtering threshold τ_{TPF}

Initialize: parameters θ
Initialize grouped dataset: $\mathcal{D}_{\text{grp}} \leftarrow \emptyset$
for $t = 1$ to M **do**
  Freeze old policy: $\pi_{\text{old}} \leftarrow \pi_\theta$ *// for ratios & KL*
  Sample rollouts (dynamic sampling):
  while $|\mathcal{D}_{\text{grp}}| < B$ **do**
    Sample one task $q \sim \mathcal{D}$
    for $j = 1$ to G **do**
      Generate rollout $o^{(j)} \sim \pi_\theta(\cdot \mid q)$ using block-diffusion decoding
      Measure speed and correctness $r_{\text{tpf}}^{(j)} \leftarrow \text{TPF}^{(j)}$, $r_{\text{acc}}^{(j)} \leftarrow c^{(j)}$
    end for
    Filter group:
    if $\left(\sum_{j=1}^{G} c^{(j)} = 0\right)$ **or** $\left(\max_j r_{\text{tpf}}^{(j)} - \min_j r_{\text{tpf}}^{(j)} < \tau_{\text{TPF}}\right)$ **then**
      Reject group *// all-wrong group and near-tied speed signal*
      continue
    end if
    Decoupled normalization within group:
      Group stats: $\mu_{\text{acc}} \leftarrow \text{mean}_j(r_{\text{acc}}^{(j)}),\ \sigma_{\text{acc}} \leftarrow \text{std}_j(r_{\text{acc}}^{(j)})$
      $\mu_{\text{tpf}} \leftarrow \text{mean}_j(r_{\text{tpf}}^{(j)}),\ \sigma_{\text{tpf}} \leftarrow \text{std}_j(r_{\text{tpf}}^{(j)})$
      Group-relative advantages $A_{\text{acc}}^{(j)} \leftarrow \dfrac{r_{\text{acc}}^{(j)} - \mu_{\text{acc}}}{\sigma_{\text{acc}} + \varepsilon}$, $A_{\text{tpf}}^{(j)} \leftarrow \dfrac{r_{\text{tpf}}^{(j)} - \mu_{\text{tpf}}}{\sigma_{\text{tpf}} + \varepsilon}$ *// per-reward decoupled normalization*
      Aggregate objectives $A^{(i,j)} \leftarrow A_{\text{acc}}^{(i,j)} + A_{\text{tpf}}^{(i,j)}$
      Group normalization $\hat{A}^{(i,j)} \leftarrow \dfrac{A^{(i,j)} - \bar{A}}{\sigma_A + \varepsilon}$
      Store accepted group $\mathcal{D}_{\text{grp}} \leftarrow (q, \{o^{(j)}\}_{j=1}^{G}, \{\hat{A}^{(j)}\}, \{c^{(j)}\})$
  end while
  Policy optimization:
  for $e = 1$ to K **do**
    Sample minibatch $\mathcal{M} \subset \mathcal{D}_{\text{grp}}$; Compute ratios $\rho^{(i,j)} \leftarrow \frac{\pi_\theta(o^{(i,j)}|q^{(i)})}{\pi_{\text{old}}(o^{(i,j)}|q^{(i)})}$
    Compute losses: $\mathcal{L} \leftarrow \mathcal{L}_{\text{PG}}(\rho, A) + \beta_{\text{KL}}\mathcal{L}_{\text{KL}}(\pi_\theta, \pi_{\text{ref}}) + w_{\text{NLL}}\mathcal{L}_{\text{NLL}}(\pi_\theta; c)$
    Update: $\theta \leftarrow \theta - \eta\nabla_\theta\mathcal{L}$
  end for
end for
Output: trained policy π_θ

C. Additional Experiments

C.1. Scalability and Efficiency Analysis

As summarized in Tab. 6, we evaluate scalability along two primary axes. First, regarding model scale: under a fixed block size of 32, scaling from 1.7B to 8B parameters significantly boosts both reasoning accuracy and parallelism. Notably, while the baseline's throughput remains stagnant as model size grows, LightningRL effectively leverages increased capacity to unlock higher parallelism. Second, regarding block size: increasing the block size on the 8B model raises the upper bound for parallel decoding, an effect especially prominent on the MBPP benchmark. Overall, LightningRL scales favorably and remains robustly effective across different model sizes and decoding configurations.

Table 6. **Comparison of SDAR and LightningRL under identical settings.** The results are grouped by Model Scale and Block Size (BS) to facilitate direct comparison across four datasets.

Scale	BS	Model	GSM8K			MATH500			MBPP			HumanEval		
			Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP	Acc (%)	TPF	AUP
1.7B	32	SDAR	71.5	2.48	176.4	41.2	5.62	226.3	39.0	3.86	149.8	48.8	2.81	136.0
		LightningRL	71.7	**3.40**	**251.4**	41.2	**6.01**	**241.2**	37.3	**5.07**	**185.8**	48.2	**3.43**	**165.2**
4B	32	SDAR	86.6	3.10	243.8	53.6	5.09	263.9	54.0	1.96	105.9	59.8	3.67	216.8
		LightningRL	85.4	**4.37**	**374.7**	56.4	**6.55**	**358.7**	52.2	**3.05**	**158.3**	57.3	**4.35**	**242.3**
8B	32	SDAR	88.9	2.85	252.5	63.6	4.81	299.5	58.0	2.44	81.1	74.4	2.39	123.8
		LightningRL	90.3	**5.58**	**492.4**	63.0	**6.28**	**407.5**	58.3	**11.10**	**641.6**	72.6	**6.30**	**450.1**
	8	SDAR	91.0	2.96	269.1	64.1	3.53	224.8	59.6	1.72	103.3	76.9	2.77	211.6
		LightningRL	89.4	**3.75**	**331.8**	67.0	**4.21**	**279.5**	58.2	**3.11**	**179.3**	76.8	**3.30**	**258.8**
	4	SDAR	91.1	2.35	213.9	71.2	2.49	176.8	63.3	1.84	116.7	78.6	1.53	120.3
		LightningRL	91.0	**3.21**	**291.6**	70.3	**3.42**	**237.7**	63.2	**2.47**	**155.5**	78.2	**2.32**	**181.3**

C.2. Wall-Clock Speed Comparison

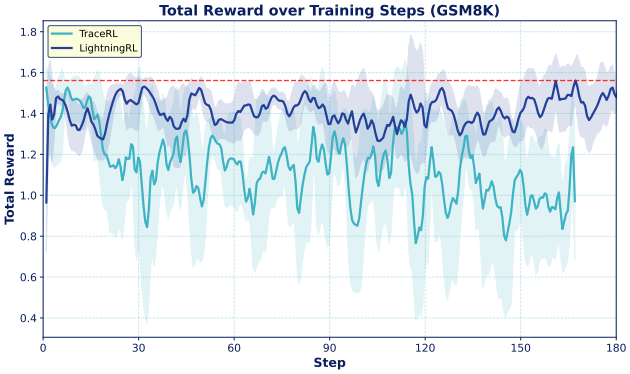
Table 7. **Tokens Per Second (TPS) performance comparison on H100 GPUs.** We report on GSM8K datasets. LightningRL achieves superior throughput.

Model	H100 TPS	TPF	Acc (%)
Qwen-2.5-7B-it	57.3	1.00	74.1
Fast-dLLM-v2	150.0	2.21	77.5
dParallel-LLaDA	172.2	5.14	72.6
d3LLM-LLaDA	<u>288.9</u>	**9.11**	73.1
SDAR	105.55	2.85	<u>88.9</u>
LightningRL	**336.03**	<u>5.58</u>	**90.3**

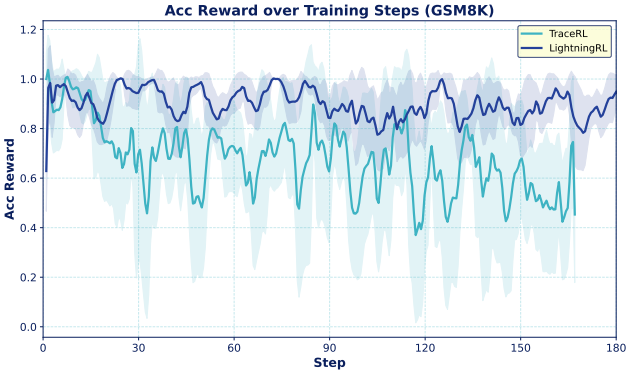
We conduct TPS benchmarks for SDAR using the SGLang on single-device H100 configurations (Tensor Parallel Size 1), with results summarized in Table 7. Since prior works are not yet natively supported by SGLang, their reported results from the original publication are cited for comparison (Qian et al., 2026). Notably, LightningRL achieves exceptional inference speed, significantly outperforming the baselines.

C.3. Training Dynamics Details

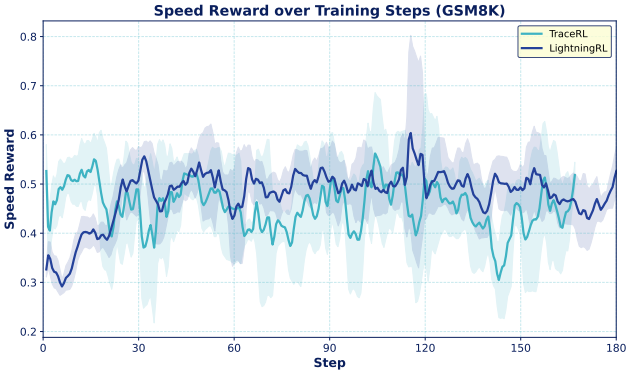
We plot the training curves of LightningRL on GSM8K in Fig. 8a-8c. For comparison, we apply TraceRL (Wang et al., 2025b), a commonly used RL framework for dLLMs, to the same training settings (including rewards, hyperparameters, etc.). As shown, during the training of TraceRL, the optimization for speed progressively erodes the correctness signal, and the accuracy reward drops sharply while speed gains remain limited. The training ultimately collapses. In contrast, LightningRL converges stably and avoids this collapse behavior, maintaining the accuracy signal while improving speed, resulting in a consistently better efficiency–accuracy frontier.



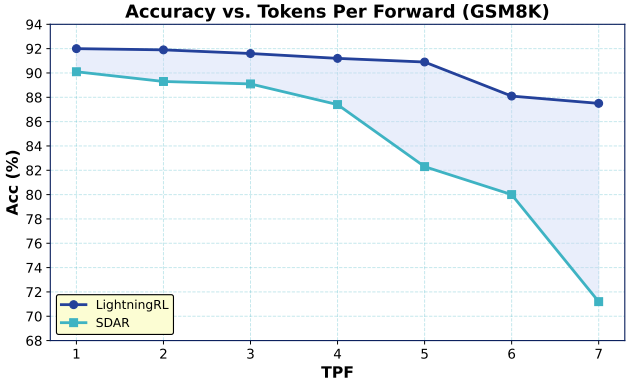
(a) **Total reward over training steps (GSM8K).**



(b) **Accuracy reward over training steps (GSM8K).**



(c) **Speed reward over training steps (GSM8K).**



(d) **Accuracy vs. TPF.** LightningRL demonstrates superior stability at higher Tokens Per Forward settings, significantly widening the accuracy gap over SDAR, which drops sharply after TPF 4.

Figure 8. **Training dynamics and Accuracy-Parallism Trad-off on GSM8K.** LightningRL avoids the objective drift and reward collapse observed in TraceRL, while sustaining higher decoding throughput and more synchronized termination.

C.4. Accuracy–Parallelism Trade-off

We further investigate the inherent trade-off between decoding parallelism and reasoning accuracy on the GSM8K benchmark. In non-autoregressive or drafting-based decoding frameworks, increasing the number of Tokens Per Forward (TPF) naturally improves throughput and parallelism. However, this aggressively challenges the model's reasoning capabilities, as it must predict longer sequences without the benefit of step-by-step autoregressive feedback, typically leading to a degradation in accuracy. As illustrated in Fig. 8d, while both methods experience accuracy degradation as TPF increases from 1 to 7, their trajectories differ significantly. The baseline SDAR is highly vulnerable to larger TPF settings; its accuracy drops precipitously after $TPF = 3$, falling from 87.4% at $TPF = 4$ to just 71.2% at $TPF = 7$. In contrast, LightningRL demonstrates remarkable robustness against this accuracy-parallelism trade-off. Its performance degrades only marginally, maintaining a high accuracy of 87.5% even at the aggressive setting of $TPF = 7$. These results confirm that LightningRL significantly pushes the Pareto frontier, mitigating the performance penalty of larger forward steps to enable higher decoding throughput without sacrificing reasoning quality.